Database
(SOC-3020)

Alessandro Agostini[1,2]

aagostini@inha.ac.kr
agostini@disi.unitn.it

[1]LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

[2]DISI, University of Trento, Italy

WEEK 4





DATABASE

Database Design: Normalization

WEEK 4

Code: W4.L1L2[LAB#2]

Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process



Any question from you?

...

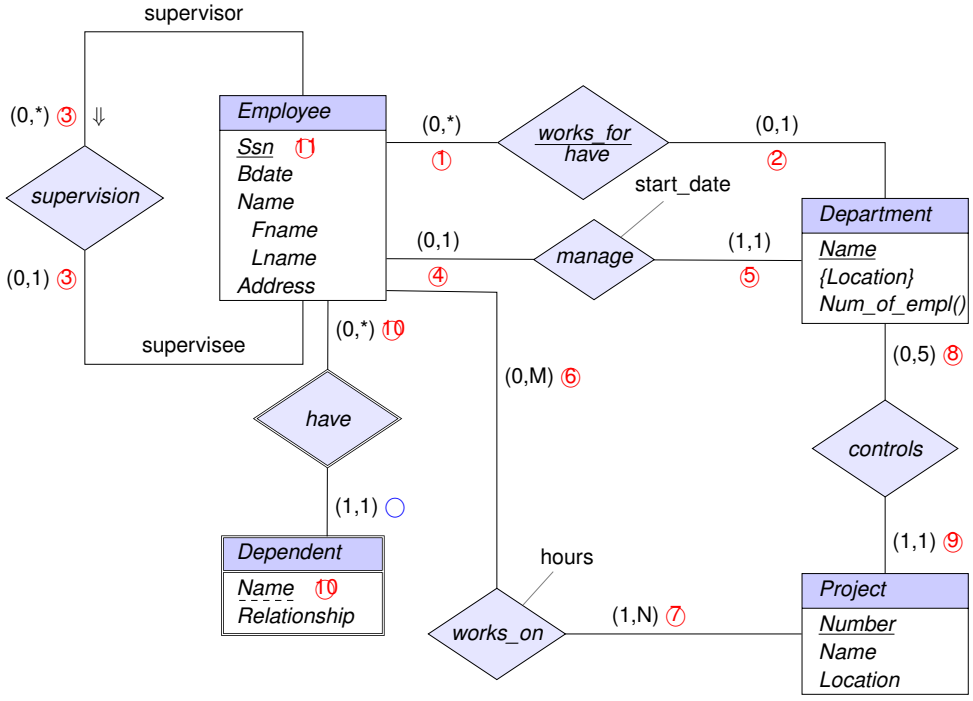
Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

Example

Company DB Conceptual Design

Company DB: E-R diagram



Outline

1 From Previous Week

2 Example: Company DB

3 Normalization Process

4 First Normal Form – 1NF

5 Second Normal Form – 2NF

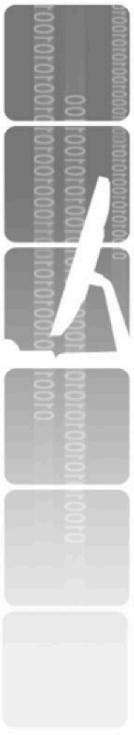
6 Third Normal Form – 3NF

7 Example (cont'): Company DB

8 LAB #2 : Normalization Process

Normalization Process

or: How to *systematically improve* logical schema design



Goals of "good" data design

– **Goal 1**: **Information preservation** : maintain all concepts, attributes, entity sets and relationship sets, generalization, specialization aggregation, etc. which are described at conceptual level (by an E-R/EE-R diagram).

– **Goal 2**: **Minimum redundancy** : minimize redundant storage and reduce the need for multiple updates to maintain consistency across multiple copies of the *same* information.



Informal Design Guidelines

- **Measures of quality for relation schemas**:
  - Clear semantics of the schema's attributes
  - Redundant information and anomalies
  - NULL values
  - Possibility to generate spurious tuples



Necessity of Normalization Theory

- So far we did not develop any measure of appropriateness or *goodness* to measure the quality of the design, other than the intuition of the designer.
- Now we discuss some of the theory that has been developed to **measure *formally*** why one set of groupings of attributes into relation schemas is better than another.
- Levels at which *goodness of relation schemas* can be discussed:
  - Logical (*from Conceptual*) level : Meaning of schemas
    - Measure of meaning is a degree of schema's **normalization**
  - Implementation level : Physical storage of schemas
    - Measure is Transaction Processing performance



Functional Dependency Theory

- Formal tool for analysis of relational schemas.
- Enables us to detect, describe and reduce some of the problems mentioned, i.e.,
  - anomalies that cause redundant data and updates;
  - waste of storage space due to NULLs;
  - generation of invalid and spurious data

  in precise (formal) terms.
- **NB**: The concept of functional dependency allows us to express *constraints on a schema* that we cannot express by using primary keys (superkeys).



Functional Dependency

- A functional dependency (FD) defines a constraint between two sets of attributes in a relation schema.

Definition. A **functional dependency on a relation schema *r* between two sets X and Y of attributes of *r***, denoted by $X \rightarrow Y$, is a constraint on the possible tuples that can form an instance of *r*. The constraint is that, for every two tuples t_1 and t_2 in an instance of *r*, if $t_1[X] = t_2[X]$, then it must also be $t_1[Y] = t_2[Y]$. (cf. Reference eBook, p. 514)

- Def: A functional dependency $X \rightarrow Y$ is *trivial* if $Y \subseteq X$.
- Def: A relation schema *r*'s instances that satisfy the FDs among the relation's attributes are called *legal relation instances* (or: legal extensions) of *r*.



Normalization

- We are now ready to use FDs to specify some aspects of the semantics of relation schemas.
- We assume that a set of FDs is given for each relation, and that each relation has a designated primary key;
- FDs + primary keys + *tests of conditions for normal forms* (see next) drives the *normalization process* of relational schema design.



Step back to FDT — Motivation

- To fix/clarify the semantics of the attributes of a logical schema R (“R” for Relation, or Table), the designer can employ FDT to define some more constrains on the schema, i.e., the **functional dependencies** on R.



Example: Update Anomalies

- Consider two schemas (cf. Reference eBook, Fig 15.3, p. 506.):

(a)

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn

FD1

FD2

(b)

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

FD1

FD2

FD3



Normalization (cont.)

- The normalization process (Codd, 1972), takes a relation schema through a series of "tests" to *certify* whether it satisfies a certain "normal form".
  - the process proceeds in a top-down fashion and suggest how decomposition of schema into new schemas of higher normal form (i.e, higher design quality).

- Codd proposed three normal forms, which he called first (1NF; 1970), second and third (2NF, 3NF; 1971) normal form.

- A stronger definition of 3NF—called Boyce-Codd normal form (BCNF)—was proposed by Boyce and Codd in 1974.





Aim of Normalization

- The aim of **normalization of data** by the *normalization process* is to achieve the desirable properties, in particular, of:

  (1) minimal redundancy, and
  (2) minimal issues with insertion, deletion, and update anomalies discussed previously (W12).

- Normalization process can be considered as a "filtering" or "purification" process to make the design have successively better quality.





Reminder: Design Guidelines

- **Measures of quality** for relation schemas:
  - Clear semantics of the schema's attributes
  - Redundant information and anomalies
  - NULL values
  - Possibility to generate spurious tuples



Normal Form - Definition

Definition. The **normal form** of a relation schema refers to the *highest normal form* condition that it meets, and hence indicates the degree to which it has been normalized.

(cf. Reference eBook, p. 517)

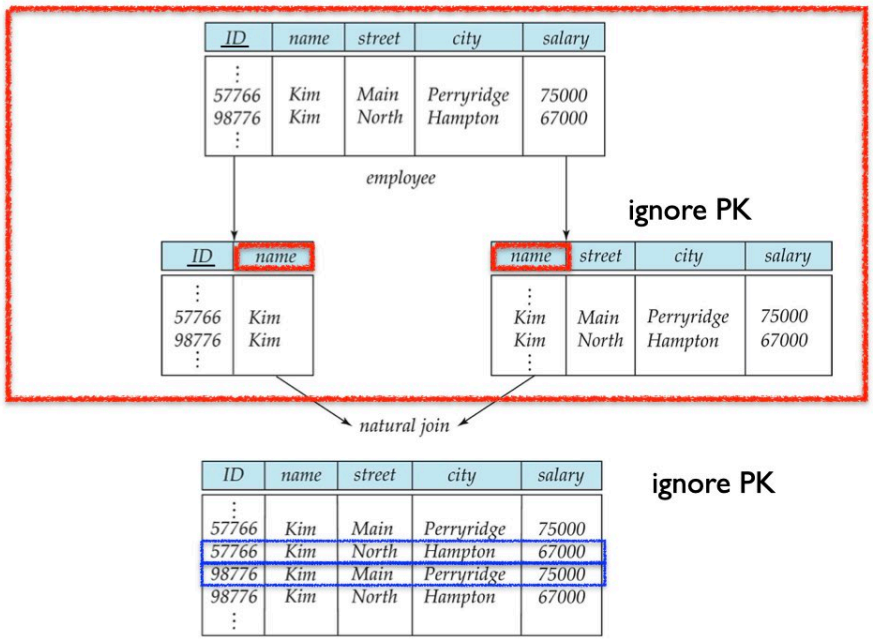


Normalization: What we look at

- Properties that the relational schemas should have after the normalization process are:
  - **Non-additive (lossless) join property**
    - guarantees that the spurious tuple generation problem does not occur with respect to the relation schemas created after decomposition by normalization.
      - Extremely critical
  - **Dependency preservation property**
    - ensures that each functional dependency is represented in some individual relation resulting after decomposition.
      - Desirable but sometimes sacrificed for other factors



Reminder: Lossy Decomposition



Practical use of Normal Forms

- Normalization carried out in practice:
  - Resulting designs are of high quality and meet the desirable properties stated previously.
  - Pays particular attention to normalization only up to 3NF and BCNF (or, at most, 4NF — we don't see it).
- **NB:** Do not need to normalize to the highest possible normal form for performance reasons.

Definition. The **de-normalization** is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form.

(cf. Reference eBook, p. 518)



Example: De-normalization

- Define FD {*dept_name*} → {*building*,*budget*} on table:

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- Observe: *instructor* and *department* not separated.



Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

Normalization Process (*cont.*)

First Normal Form – 1NF

or: Logical design of composite and multi-valued attributes



First Normal Form (1NF)

- Now a part of the formal definition of a relation schema in the relational model; but historically defined to disallow multivalued attributes or composite attributes, and their combinations.

- It states that:
  - the **domain of an attribute must include only *atomic*** (simple, indivisible) ***values*** and
  - the value of any attribute in a tuple must be a *single value* from the domain of that attribute.



Example

- Consider the following relation schema:

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(cf. Reference eBook, Fig. 15.9)

- As you can see from the schema instance, the schema is not in 1NF.
- Values of *Dlocations* are not atomic (but sets).



First Normal Form (cont.)

- Techniques to achieve first normal form:

1. Remove attribute and place in separate relation.
2. Expand the primary key to include the attribute.
3. Use several atomic attributes (cf. eBook, p. 521).



Example

- Remove non-atomic attribute and place in a new, separate relation schema:

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

(cf. Reference eBook, Fig. 15.2)

these are in 1NF



First Normal Form (cont.)

- It does not allow composite attributes (sometimes also called: **nested relations**).
- **Example**:

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)})

- To change the schema into a 1NF:
  - Remove nested relation attributes into new set of simple attributes: Projs-Pnumber, Projs-Hours.



Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

Normalization Process (*cont.*)

Second Normal Form – 2NF

or: Eliminate *Partial* Functional Dependencies on Primary Key

Second Normal Form (2NF)

- Based on the "technical concept" of ***full*** **functional dependency** of attributes.

Definition. A functional dependency $X \rightarrow Y$ is a **full functional dependency** if removal of any attribute A from X means that the dependency does not hold any more.

Definition. A functional dependency $X \rightarrow Y$ is a **partial functional dependency** if some attribute A in X can be removed from X and the dependency still holds.

(cf. Reference eBook, p. 523)



Example

- Consider the following relation schema:

(cf. Reference eBook, Fig. 15.3)

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

FD1

FD2

FD3

- FD1: {*Ssn*,*Pnumber*} → *Hours* full dependency
  (*Ssn* → *Hours* and *Pnumber* → *Hours* don't hold).
- {*Ssn*,*Pnumber*} → *Ename* partial dependency
  because FD2: *Ssn* → *Ename*



Prime Attributes

- **Prime vs nonprime attribute:**

Definition. An attribute of relation schema *R* is called a **prime attribute** of *R* if it is a member of some candidate* key of *R*. An attribute is called **nonprime** if it is not a prime attribute—that is, if it is not a member of any candidate key.

* the definition is general. Here we only need it to be primary key.

(cf. Reference eBook, p. 519)



Example

- Consider the following relation schema:

WORKS_ON

Ssn	Pnumber	Hours

P.K.

(cf. Reference eBook, Fig. 15.1)

- *Ssn* and *Pnumber* are prime attributes of WORKS_ON, whereas *Hours* is nonprime.



Second Normal Form (cont.)

Definition (Codd, 1971) A relation schema *R* is in second normal form (2NF) if every nonprime attribute A in *R* is *fully functionally dependent* on the primary* key of *R*.

(cf. Reference eBook, p. 523)

note this
REMARK:
* A **more general definition of 2NF** generalises it to "on every *candidate key.*" (We don't need such generalization.)



Example

- Consider the following relation schema R:

EMP_PROJ (cf. Reference eBook, Fig. 15.3)

Ssn	Pnumber	Hours	Ename	Pname	Plocation

FD1
FD2
FD3

- Note the primary key. Then, R is not in 2NF:
  - {*Ssn*,*Pnumber*} → *Ename* partial dependency
    - FD2: *Ssn* → *Ename* holds. Also:
  - {*Ssn*,*Pnumber*} → *Pname* partial dependency
    - FD3: *Pnumber* → {*Pname*,*Plocation*} holds



Second Normal Form (cont.)

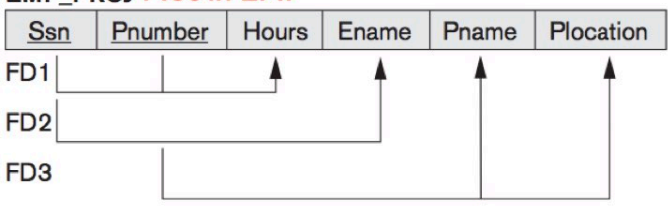
- If a relation schema is not in 2NF, it can be *2NF-normalized* into a number of 2NF relations
- To do it, look at the FDs and decompose the schema into components where nonprime attributes satisfy the 2NF condition.
  - In particular:

nonprime attributes must be associated only with the part of the primary key on which they are **fully** functionally dependent.

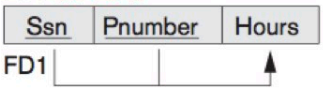


Normalization to 2NF

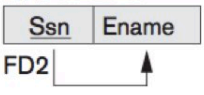
EMP_PROJ Not in 2NF



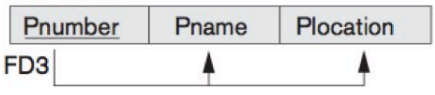
EP1 In 2NF



EP2 In 2NF



EP3 In 2NF



In short: eliminate the partial dependencies on primary key of all nonprime attributes

(cf. Reference eBook, Fig. 15.11)





Reminder on Normalization

- Properties that the relational schemas should have after the normalization process are:
  - **Non-additive (lossless) join property**
    - guarantees that the spurious tuple generation problem does not occur with respect to the relation schemas created after decomposition by normalization.
      - Extremely critical
  - **Dependency preservation property**
    - ensures that each functional dependency is represented in some individual relation resulting after decomposition.
      - Desirable but sometimes sacrificed for other factors



Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

Normalization Process (*cont.*)

Third Normal Form – 3NF

or: Eliminate *Transitive* Functional Dependencies on Primary Key



Transitive Dependency

- Based on the "technical concept" of **transitive dependency** of attributes.

Definition. A functional dependency $X \rightarrow Y$ in a relation schema R is a **transitive dependency** if there exists a set of attributes Z in R that is neither a candidate* key nor a subset of any* key of R, and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

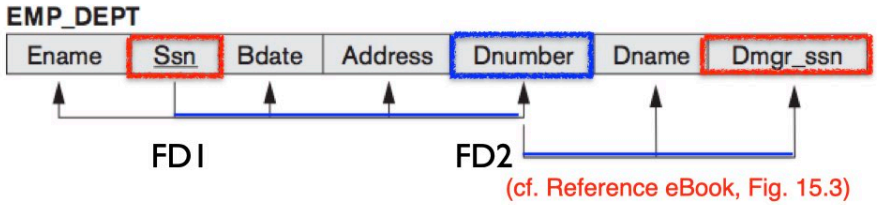
* the definition is general. We only need it to be primary key.

(cf. Reference eBook, p. 523)



Example

- Consider the following relation schema:



(cf. Reference eBook, Fig. 15.3)

- {*Ssn*} → *Dmgr_ssn* is a transitive dependency:

1. {*Ssn*} → *Dmgr_ssn* inferred by Transitivity;
2. *Dnumber* is not a part of any primary key*
   * more generally: any super key.





Another Example

- Suppose {*dept_name*} → {*building*,*budget*} on table:

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

transitively dependent on ID (primary key)

- Observe 1: primary key is ambiguous (incomplete)
- Observe 2: *dept_name* depends on *ID* (*ID* is PK)



Third Normal Form (3NF)

Definition. According to Codd's original definition, a relation schema *R* is in 3NF if it satisfies 2NF *and* no nonprime attribute of R is *transitively dependent* on the primary key.

(cf. Reference eBook, p. 524)

Definition (more general (*) - we see it later). A relation schema R is in 3NF if, whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R, either (a) *X* is a superkey of R or (b) *A* is a prime attribute of R.

(cf. Reference eBook, p. 528)

(*) "more general" because it does *not* use the notion of 2NF.



Example: Normalization to 3NF

Not in 3NF:

transitively dependent on Ssn (primary key)

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn

FD1 FD2

In 3NF:

ED1

Ename	Ssn	Bdate	Address	Dnumber

FD1

In 3NF:

ED2

Dnumber	Dname	Dmgr_ssn

FD2

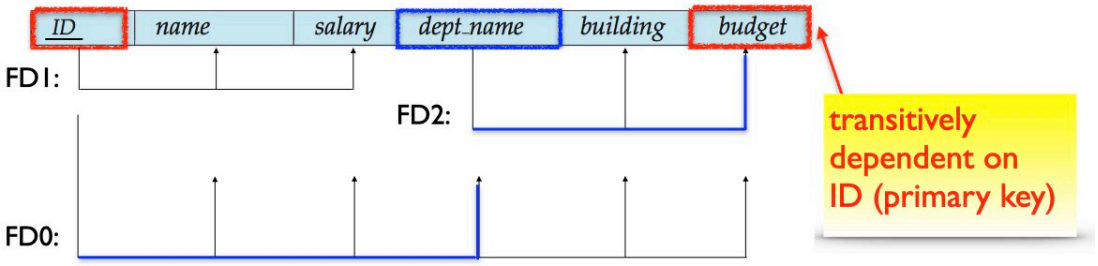
(cf. Reference eBook, Fig. 15.11)





Another Example

- Suppose PK, FD1, and FD2 are defined on table:



- Observe 1: primary key is ambiguous (incomplete)
- Observe 2: FD0 holds because of *ID* is PK.





3NF (general definition)

- Note: $X \rightarrow Y$ is *trivial* if $Y \subseteq X$.

Definition (more general (*)). A relation schema R is in 3NF if, whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R, either (a) X is a superkey of R or (b) A is a prime attribute of R.

(cf. Reference eBook, p. 528)

(*) because it does *not* use the notion of 2NF.





Example: 3NF (general definition)

- Suppose {*dept_name*} → {*building*,*budget*} on table:

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

not prime
not (primary)key

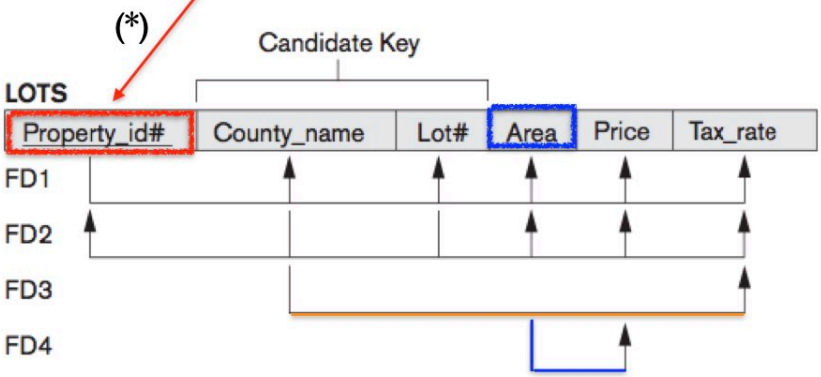
- Observe: {*dept_name*} → {*building*,*budget*} *non trivial*
- Observe: *dept_name* not PK and *budget* not prime





Example: Normalization to 3NF

Not in 3NF (but in 2NF; not in general 2NF because of FD3):



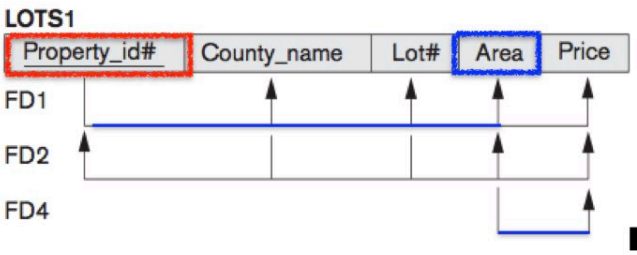
(*) Note the single attribute

(cf. Reference eBook, Fig. 15.12)

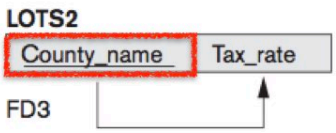


Example (cont.)

LOTS1 not in 3NF (but in 2NF):

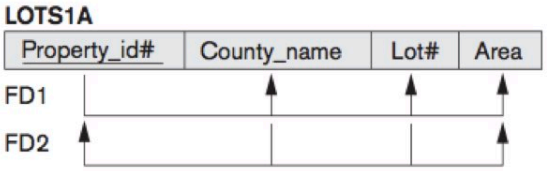


In 3NF (and 2NF):



LOTS1 normalized to 3NF

In 3NF:



In 3NF:



(cf. Reference eBook, Fig. 15.12)



3NF Normalization: Remark

- Normalization procedure to 3NF preserves the dependencies defined on the input schema.
- Remember: <u>after the normalization process</u>, the resulting schema(s) satisfy these properties:
  - **Non-additive (lossless) join property**
    - guarantees that the spurious tuple generation problem does not occur with respect to the relation schemas created after decomposition by normalization.
      - Extremely critical
  - **Dependency preservation property**
    - ensures that each functional dependency is represented in some individual relation resulting after decomposition.
      - Desirable but sometimes sacrificed for other factors



Summary of Normal Forms 1st to 3rd

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

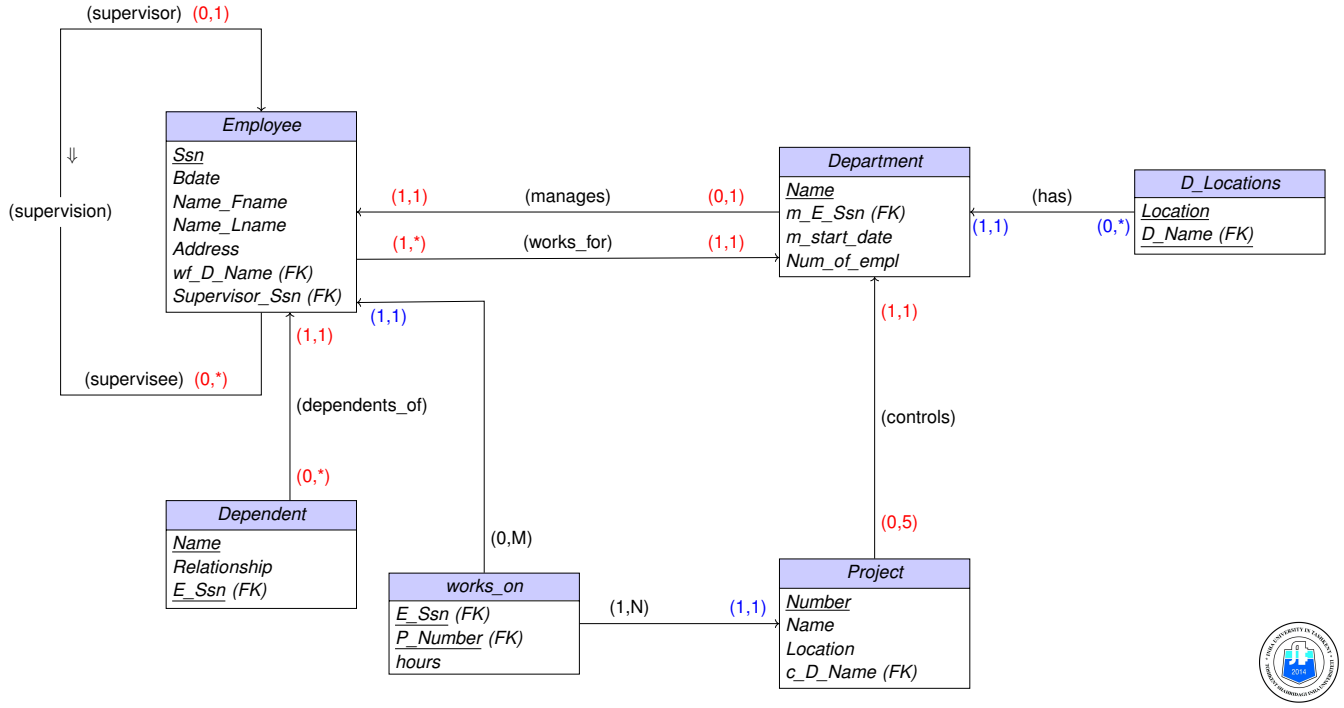
(cf. Reference eBook, p. 525)



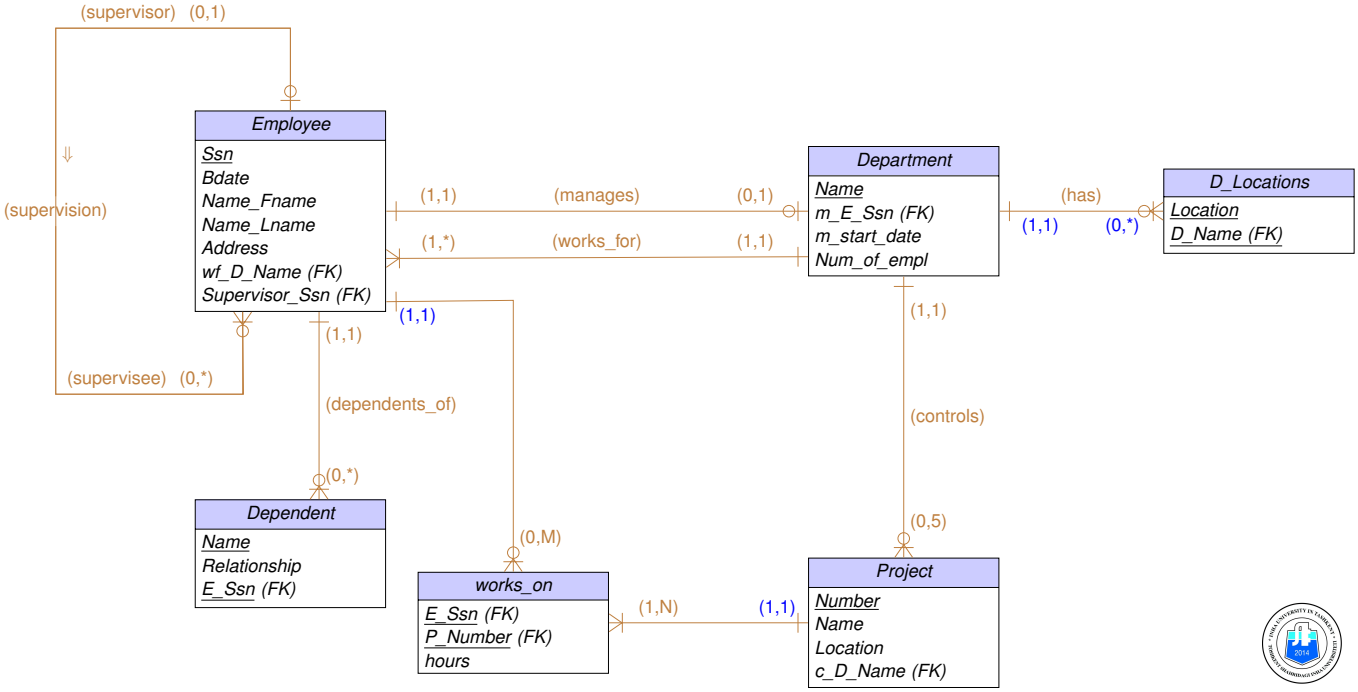
Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

3NF-Normalized Logical Schema (tagged with (*min*, *max*) contraints)



3NF-Normalized Logical Schema: "Crow's Foot" Notation



Outline

1. From Previous Week
2. Example: Company DB
3. Normalization Process
4. First Normal Form – 1NF
5. Second Normal Form – 2NF
6. Third Normal Form – 3NF
7. Example (cont'): Company DB
8. LAB #2 : Normalization Process

LAB #1: *Logical* Data Design*

* Participation checked (2-3 members per working group)

In class only (no slides provided).



Summary

1 From Previous Week

2 Example: Company DB

3 Normalization Process

4 First Normal Form – 1NF

5 Second Normal Form – 2NF

6 Third Normal Form – 3NF

7 Example (cont'): Company DB

8 LAB #2 : Normalization Process



DATABASE

End of Slides

WEEK 4

Code: W4.L1L2[LAB#2]